

Insecure Direct Object References

Insecure Direct Object References



Show hints Reset lesson

1 2 3 4 5 6

Authenticate First, Abuse Authorization Later

Many access control issues are susceptible to attack from an authenticated-but-unauthorized user. So, let's start by legitimately authenticating. Then, we will look for ways to bypass or abuse Authorization.

The id and password for the account in this case are 'tom' and 'cat' (It is an insecure app, right?).

After authenticating, proceed to the next screen.

user/pass user: pass:

Login with the provided credentials tom and cat

Show hints Reset lesson

1 2 3 4 5 6

Authenticate First, Abuse Authorization Later

Many access control issues are susceptible to attack from an authenticated-but-unauthorized user. So, let's start by legitimately authenticating. Then, we will look for ways to bypass or abuse Authorization.

The id and password for the account in this case are 'tom' and 'cat' (It is an insecure app, right?).

After authenticating, proceed to the next screen.

✓
user/pass user: pass:

You are now logged in as tom. Please proceed.

Insecure Direct Object References



[Show hints](#) [Reset lesson](#)

1 2 3 4 5 6

Observing Differences & Behaviors

A consistent principle from the offensive side of AppSec is to view differences from the raw response to what is visible. In other words (as you may have already noted in the client-side filtering lesson), there is often data in the raw response that doesn't show up on the screen/page. View the profile below and take note of the differences.

[View Profile](#)

In the text input below, list the two attributes that are in the server's response, but don't show above in the profile.

[Submit Diffs](#)

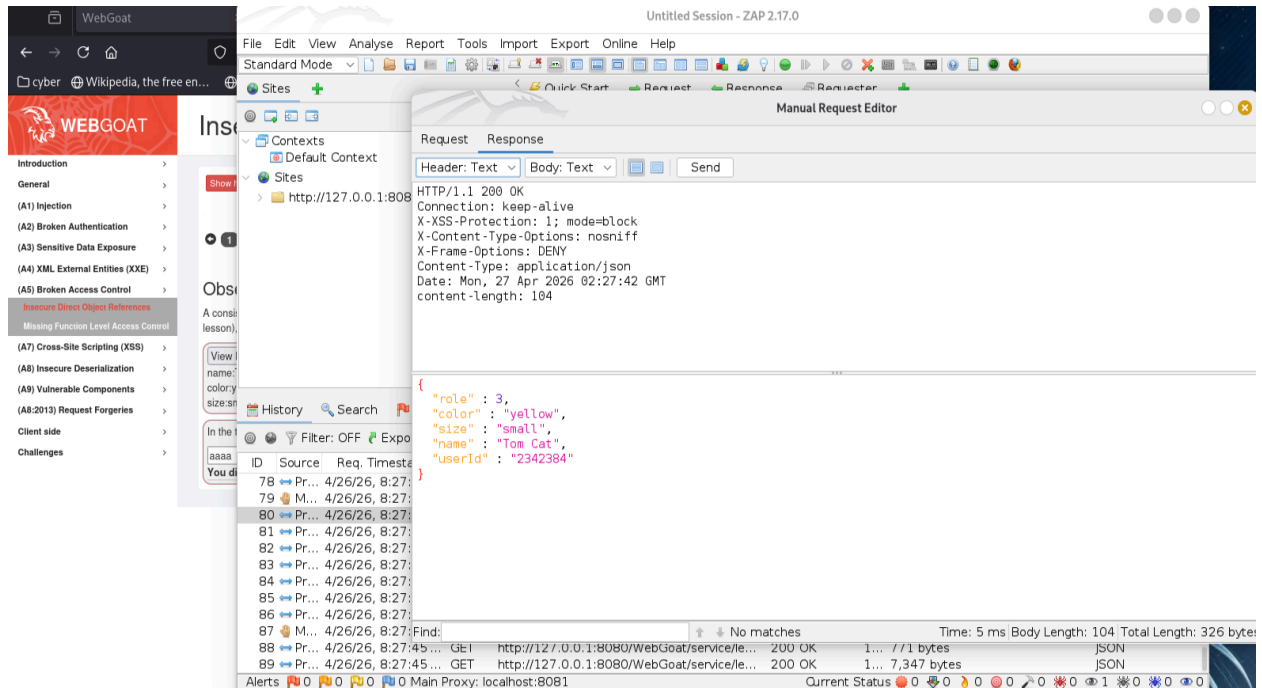
Click view profile

[View Profile](#)
name:Tom Cat
color:yellow
size:small

In the text input below, list the two attributes that are in the server's response, but don't show above in the profile.

[Submit Diffs](#)

Start ZAP and intercept the request to view profile, send the request



We can see two extra attributes: role, and userID

Insecure Direct Object References



[Show hints](#) [Reset lesson](#)

1 2 3 4 5 6

Observing Differences & Behaviors

A consistent principle from the offensive side of AppSec is to view differences from the raw response to what is visible. In other words (as you may have already noted in the client-side filtering lesson), there is often data in the raw response that doesn't show up on the screen/page. View the profile below and take note of the differences.

[View Profile](#)
name:Tom Cat
color:yellow
size:small

☒ In the text input below, list the two attributes that are in the server's response, but don't show above in the profile.
 [Submit Diffs](#)
Correct, the two attributes not displayed are userID & role. Keep those in mind

Insecure Direct Object References



[Show hints](#) [Reset lesson](#)

➕

1

2

3

4

5

6

➖

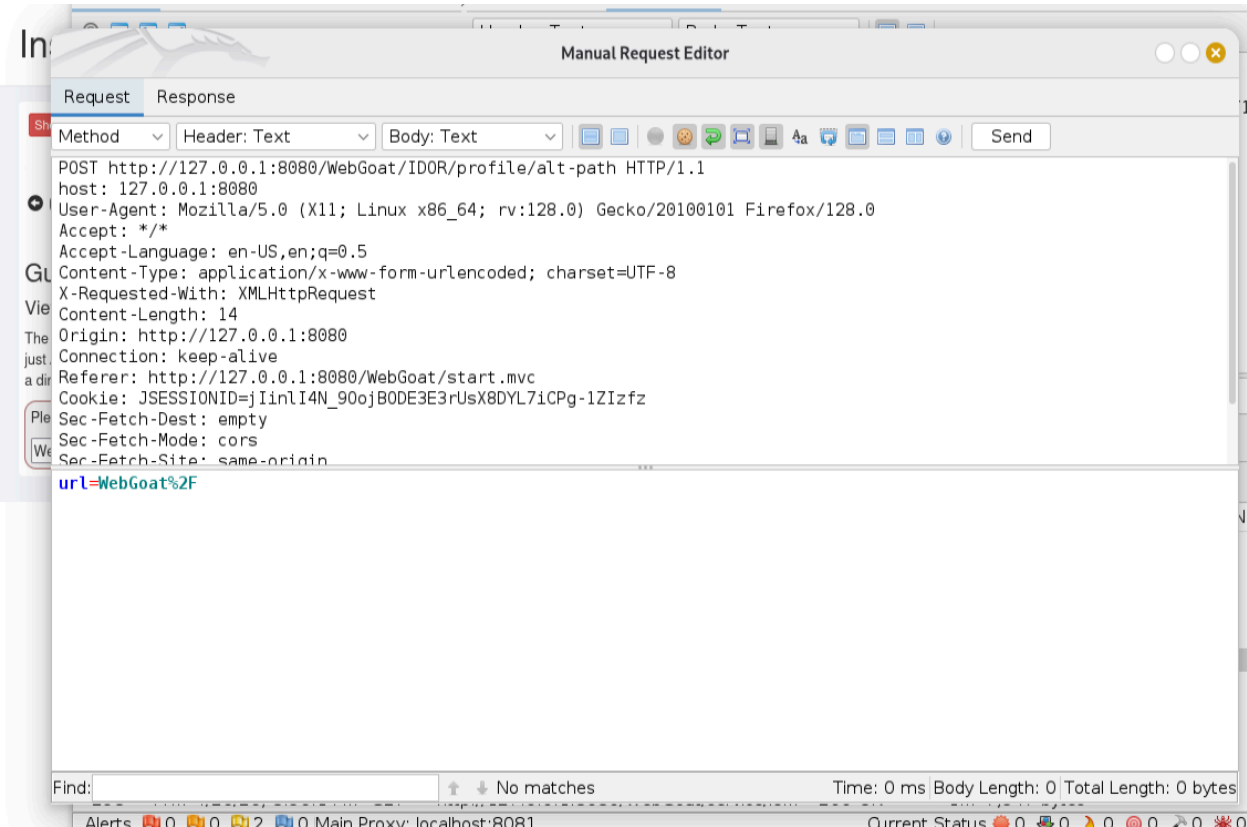
Guessing & Predicting Patterns

View Your Own Profile Another Way

The application we are working with seems to follow a RESTful pattern so far as the profile goes. Many apps have roles in which an elevated user may access content of another. In that case, just /profile won't work since the own user's session/authentication data won't tell us whose profile they want view. So, what do you think is a likely pattern to view your own profile explicitly using a direct object reference?

Please input the alternate path to the Url to view your profile. Please start with 'WebGoat' (i.e. disregard 'http://localhost:8080/')

Click submit to intercept with ZAP



Notice the path, we can use the userID from the previous challenge in that path

WebGoat/IDOR/profile/2342384

Insecure Direct Object References



Show hints Reset lesson

1 2 3 4 5 6

Guessing & Predicting Patterns

View Your Own Profile Another Way

The application we are working with seems to follow a RESTful pattern so far as the profile goes. Many apps have roles in which an elevated user may access content of another. In that case, just /profile won't work since the own user's session/authentication data won't tell us whose profile they want view. So, what do you think is a likely pattern to view your own profile explicitly using a direct object reference?



Please input the alternate path to the Url to view your own profile. Please start with 'WebGoat' (i.e. disregard 'http://localhost:8080/')

/3oat/IDOR/profile/2342384 Submit

Congratulations, you have used the alternate Url/route to view your own profile.

{role=3, color=yellow, size=small, name=Tom Cat, userId=2342384}

Insecure Direct Object References



Show hints Reset lesson

1 2 3 4 5 6

Playing with the Patterns

View Another Profile

View someone else's profile by using the alternate path you already used to view your own profile. Use the 'View Profile' button and intercept/modify the request to view another profile. Alternatively, you may also just be able to use a manual GET request with your browser.

View Profile

Edit Another Profile

Older apps may follow different patterns, but RESTful apps (which is what's going on here) often just change methods (and include a body or not) to perform different functions.

Use that knowledge to take the same base request, change its method, path and body (payload) to modify another user's (Buffalo Bill's) profile. Change the role to something lower (since higher privilege roles and users are usually lower numbers). Also change the user's color to 'red'.

View Profile

Intercept the request to view profile

Untitled Session - ZAP 2.17.0

File Edit View Analyse Report Tools Import Export Online Help

Standard Mode

Sites + Quick Start Request Response Requester +

Header: Text Body: Text

GET http://127.0.0.1:8080/WebGoat/IDOR/profile/%7BuserId%7D HTTP/1.1
 host: 127.0.0.1:8080
 User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:128.0) Gecko/20100101 Firefox/128.0
 Accept: */*
 Accept-Language: en-US,en;q=0.5
 Content-Type: application/x-www-form-urlencoded; charset=UTF-8
 X-Requested-With: XMLHttpRequest
 Connection: keep-alive
 Referer: http://127.0.0.1:8080/WebGoat/start.mvc
 Cookie: JSESSIONID=j1inLI4N_90ojB0DE3E3rUsX8DYL7iCPg-1Z1zfz
 Sec-Fetch-Dest: empty
 Sec-Fetch-Mode: cors
 Sec-Fetch-Site: same-origin
 Priority: u=0
 content-length: 0

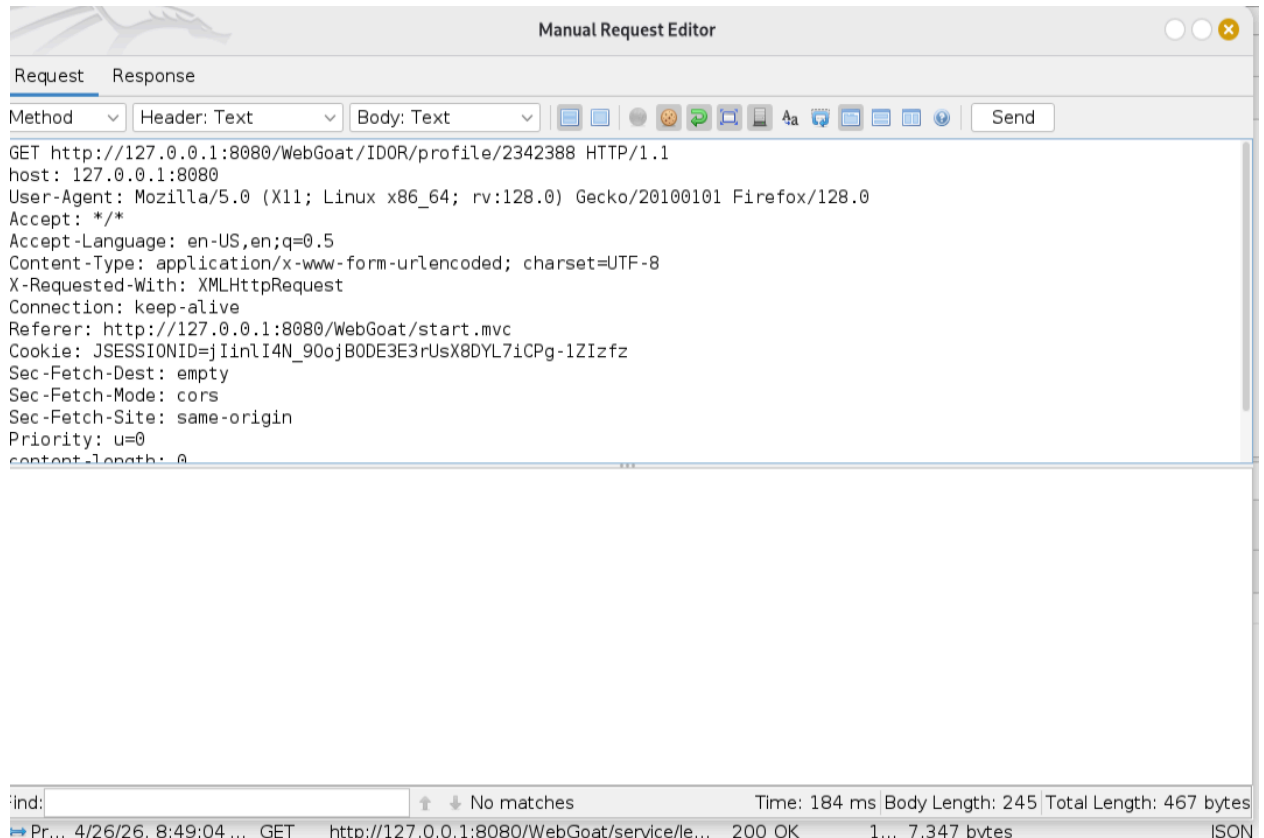
History Search Alerts Output +

Filter: OFF Export

ID	Source	Req. Timestamp	Meth...	URL	Code	Reason	RTT	Size Resp. Bo...	Highest Al...	N...	Tags
716	Pr...	4/26/26, 8:48:47 ...	GET	http://127.0.0.1:8080/WebGoat/service/le...	200	OK	1...	769 bytes			JSON
717	Pr...	4/26/26, 8:48:48 ...	GET	http://127.0.0.1:8080/WebGoat/IDOR/profi...	500	Internal ...	4...	11,425 bytes			JSON
718	Pr...	4/26/26, 8:48:49 ...	GET	http://127.0.0.1:8080/WebGoat/service/le...	200	OK	1...	7,347 bytes			JSON
719	Pr...	4/26/26, 8:48:52 ...	GET	http://127.0.0.1:8080/WebGoat/service/le...	200	OK	1...	769 bytes			JSON
720	Pr...	4/26/26, 8:48:54 ...	GET	http://127.0.0.1:8080/WebGoat/service/le...	200	OK	1...	7,347 bytes			JSON
721	Pr...	4/26/26, 8:48:57 ...	GET	http://127.0.0.1:8080/WebGoat/service/le...	200	OK	1...	769 bytes			JSON
722	Pr...	4/26/26, 8:48:59 ...	GET	http://127.0.0.1:8080/WebGoat/service/le...	200	OK	1...	7,347 bytes			JSON
723	Pr...	4/26/26, 8:49:02 ...	GET	http://127.0.0.1:8080/WebGoat/service/le...	200	OK	1...	769 bytes			JSON
724	Pr...	4/26/26, 8:49:04 ...	GET	http://127.0.0.1:8080/WebGoat/service/le...	200	OK	1...	7,347 bytes			JSON
725	M...	4/26/26, 8:49:05 ...	GET	http://127.0.0.1:8080/WebGoat/IDOR/profi...	500	Internal ...	5...	11,425 bytes			JSON
726	Pr...	4/26/26, 8:49:07 ...	GET	http://127.0.0.1:8080/WebGoat/service/le...	200	OK	1...	769 bytes			JSON
727	Pr...	4/26/26, 8:49:09 ...	GET	http://127.0.0.1:8080/WebGoat/service/le...	200	OK	1...	7,347 bytes			JSON

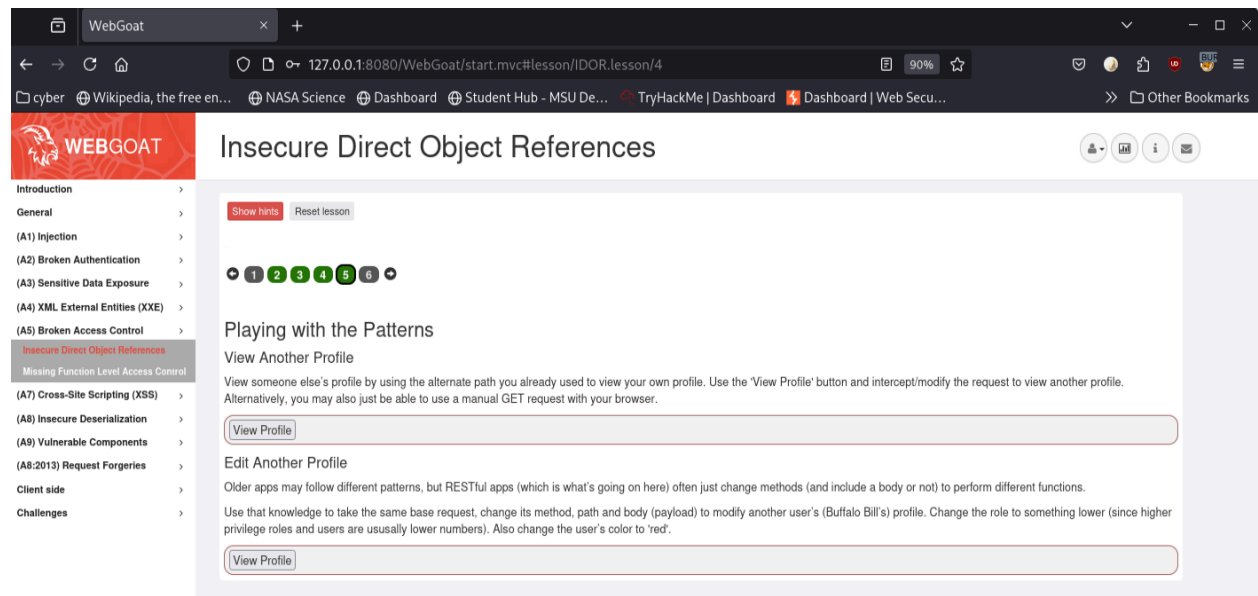
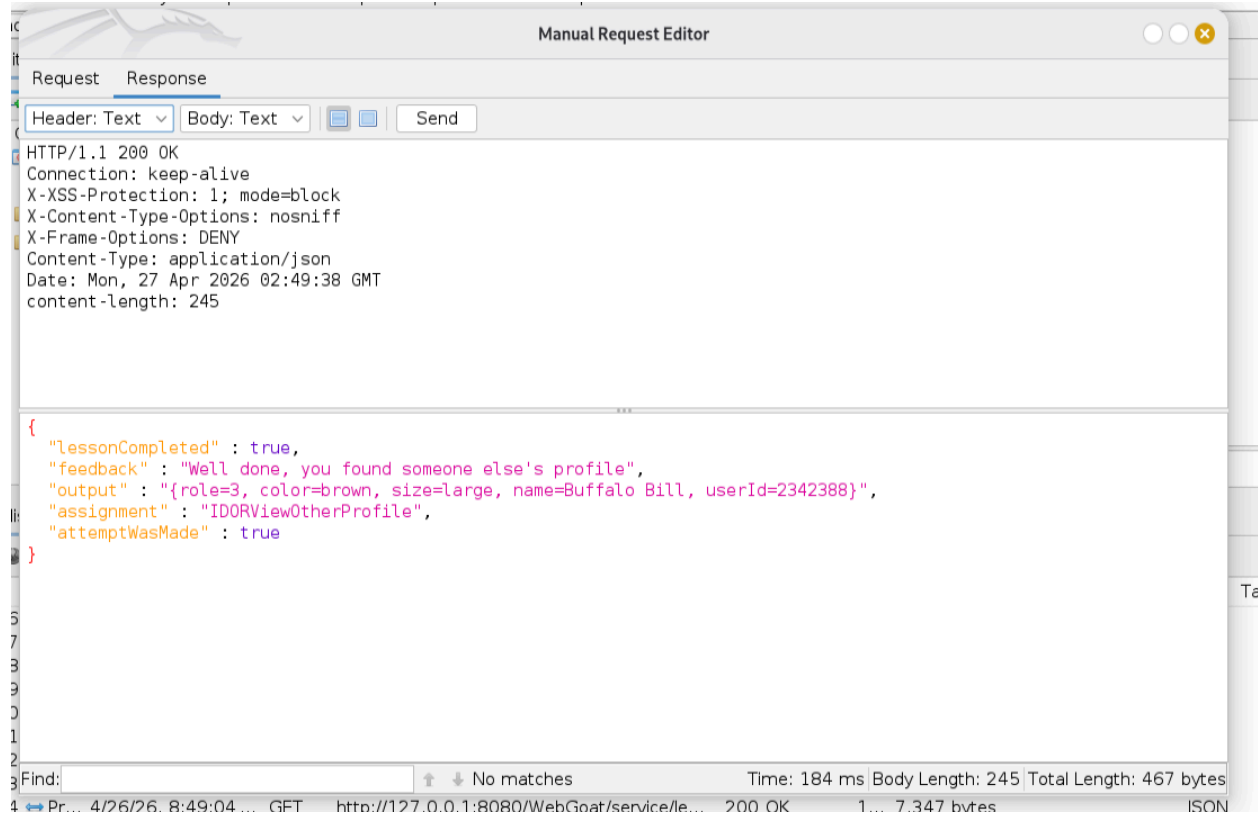
Alerts 0 2 4 5 Main Proxy: localhost:8081 Current Status 0

incrementing the userID last digit by one until another profile is found



The path /WebGoat/IDOR/profile/2342388

Works and we can send the request and get a 200 response



Missing Function Level Access Control

[Show hints](#) [Reset lesson](#)

➔ 1 2 3 ➔

Relying on Obscurity

One could rely on HTML, CSS or javascript to hide links that users don't normally access. In the past there has been a case where a network router tried to protect (hide) admin functionality with javascript in the UI: <https://www.wired.com/2009/10/routers-still-vulnerable>.

Finding Hidden Items

There are usually hints to finding functionality the UI does not openly expose in ...

- HTML or javascript comments
- Commented out elements
- Items hidden via CSS controls/classes

Your Mission

Find two invisible menu items in the menu below that are, or would be, of interest to an attacker/malicious user and submit the labels for those menu items (there are no links right now in the menus).

Account

My Profile

Messages

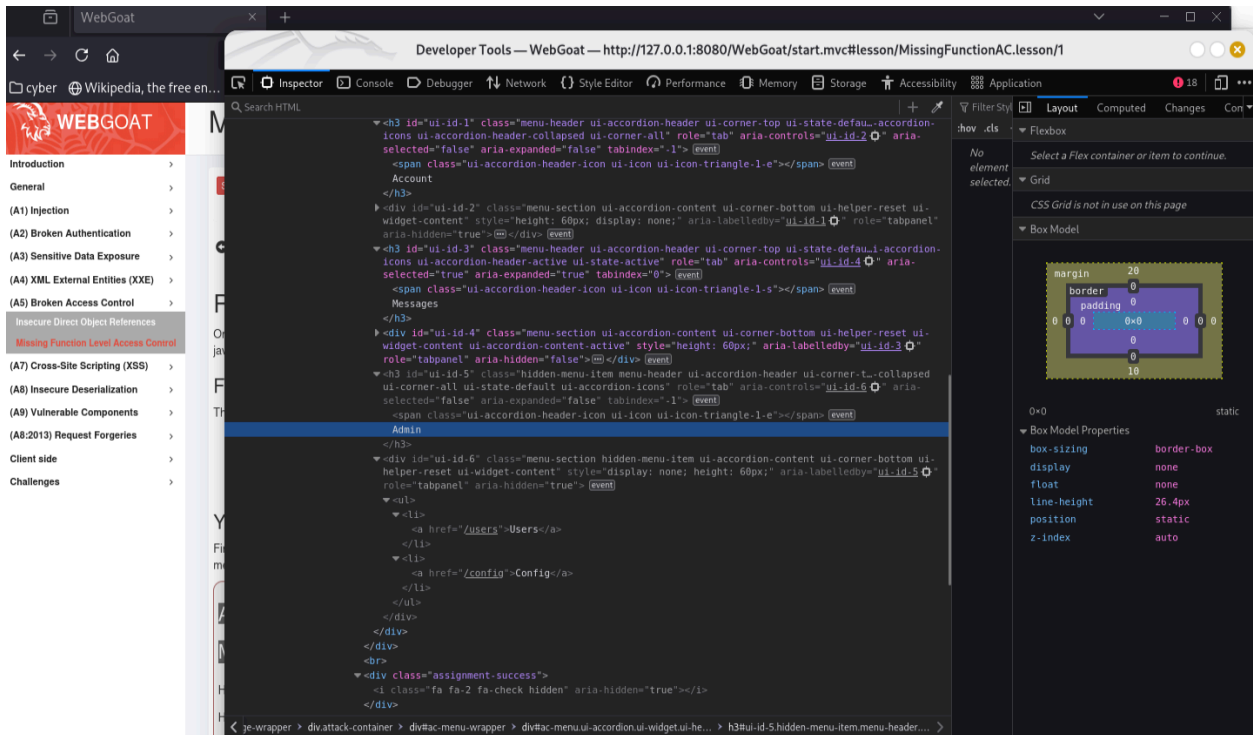
Log Out

Hidden Item 1

Hidden Item 2

Submit

Inspect the source and find the hidden menu items under the section



“Users”
“Config”



Relying on Obscurity

One could rely on HTML, CSS or javascript to hide links that users don't normally access. In the past there has been a case where a network router tried to protect (hide) admin functionality with javascript in the UI: <https://www.wired.com/2009/10/routers-still-vulnerable>.

Finding Hidden Items

There are usually hints to finding functionality the UI does not openly expose in ...

- HTML or javascript comments
- Commented out elements
- Items hidden via CSS controls/classes

Your Mission

Find two invisible menu items in the menu below that are, or would be, of interest to an attacker/malicious user and submit the labels for those menu items (there are no links right now in the menus).

Account

Messages

Unread Messages (3)

✓ Compose Message

Hidden Item 1

Hidden Item 2

Submit

Correct! And not hard to find are they!? One of these urls will be helpful in the next lab.

Missing Function Level Access Control



Show hints Reset lesson

1 2 3

Just Try It

As the previous page described, sometimes applications rely on client-side controls to control access (obscurity). If you can find items which are invisible, just try them and see what happens. Yes, it can be that simple!

Gathering User Info

Often data dumps originate from vulnerabilities such as sql injection, but they can also come from poor or lacking access control.

It will likely take multiple steps and multiple attempts to get this one. Pay attention to the comments and leaked info. And you'll need to do some guessing too. You may need to use another browser/account along the way. Start with the info you already gathered (hidden menu items) to see if you can pull the list of users and then provide the 'Hash' for your own user account.

Your Hash:

Submit

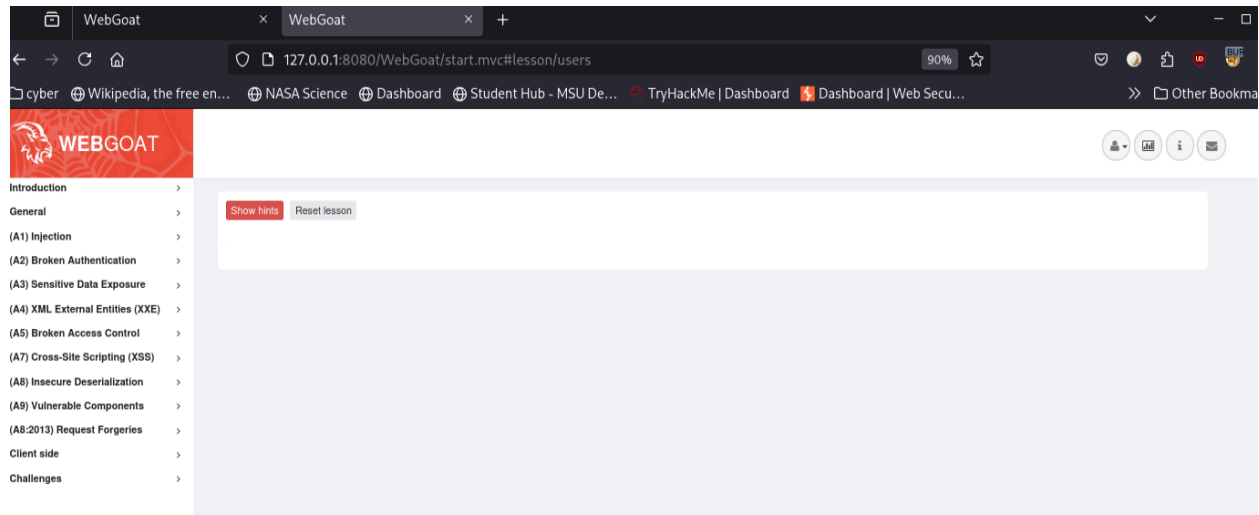
Intercept a request from submit

The screenshot shows the ZAP 2.17.0 Manual Request Editor window. The request is a POST to `http://127.0.0.1:8080/WebGoat/access-control/user-hash`. The headers include `Host: 127.0.0.1:8080`, `User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:128.0) Gecko/20100101 Firefox/128.0`, `Accept: */*`, `Accept-Language: en-US,en;q=0.5`, `Content-Type: application/x-www-form-urlencoded; charset=UTF-8`, `X-Requested-With: XMLHttpRequest`, `Content-Length: 9`, `Origin: http://127.0.0.1:8080`, `Connection: keep-alive`, `Referer: http://127.0.0.1:8080/WebGoat/start.mvc`, `Cookie: JSESSIONID=jIinLI4N_90ojB0DE3E3rUsX8DYL71CPg-1Z1zfz`, `Sec-Fetch-Mode: empty`, `Sec-Fetch-Site: same-origin`, and `userHash=`.

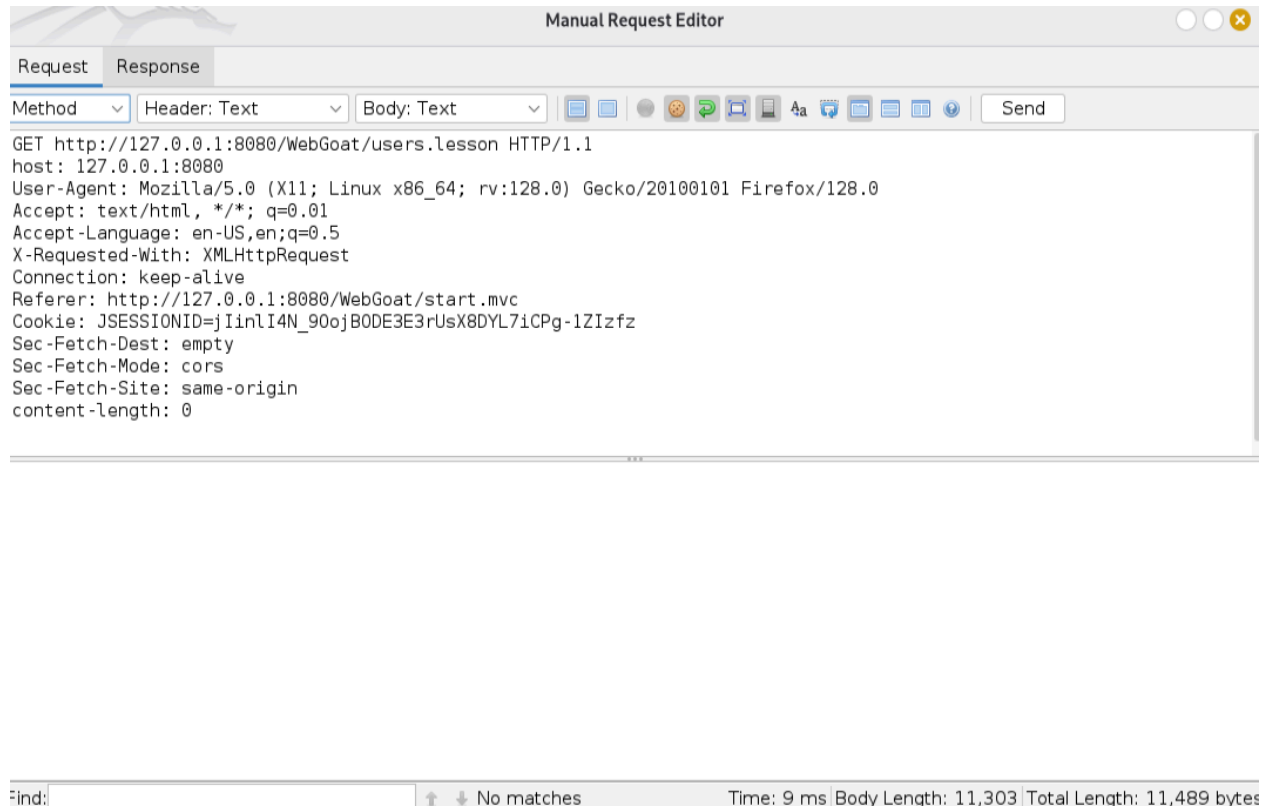
Find:	Time: 0 ms	Body Length: 0	Total Length: 0 bytes
1,0... → Pr... 4/26/26, 9:03:15 ... GET http://127.0.0.1:8080/webGoat/service/e... 200 OK 1... 336 bytes			
1,0... → Pr... 4/26/26, 9:03:15 ... GET http://127.0.0.1:8080/WebGoat/service/e... 200 OK 1... 7,347 bytes			

Visit the endpoint from the previous challenge:

<http://127.0.0.1:8080/WebGoat/start.mvc#lesson/users>



Intercept a GET packet



From the hint, we know we should change the content-type header